Python Code

```
"""
Sports Analytics
"""

import numeric
import codeskulptor
from urllib import request
import comp140_module6 as sports

def read_matrix(filename):
    """
    Parse data from the file with the given filename into a matrix.

    input:
        - filename: a string representing the name of the file

    returns: a matrix containing the elements in the given file
    """
    # initialize and load file
    values = []
    url = codeskulptor.file2url(filename)
    netfile = request.urlopen(url)

    # go through file and convert to str
    for line in netfile.readlines():
        line = line.decode('utf-8').replace(',', ' ').split()
        values.append([float(x) for x in line])

    # list to matrix
    matrix = numeric.Matrix(values)
    return matrix

class LinearModel:
    """
    A class used to represent a Linear statistical
    model of multiple variables. This model takes
    a vector of input variables and predicts that
    the measured variable will be their weighted sum.
    """

    def __init__(self, weights):
        """
        Create a new LinearModel.

        inputs:
            - weights: an m x 1 matrix of weights
        """
        self._weights = weights

    def __str__(self):
```

```
        """
        Return: weights as a human readable string.
        """
        return str(self._weights)

    def get_weights(self):
        """
        Return: the weights associated with the model.
        """
        return self._weights

    def generate_predictions(self, inputs):
        """
        Use this model to predict a matrix of
        measured variables given a matrix of input data.

        inputs:
            - inputs: an n x m matrix of explanatory variables

        Returns: an n x 1 matrix of predictions
        """
        #simple formula
        return inputs @ self._weights

    def prediction_error(self, inputs, actual_result):
        """
        Calculate the MSE between the actual measured
        data and the predictions generated by this model
        based on the input data.

        inputs:
            - inputs: inputs: an n x m matrix of explanatory variables
            - actual_result: an n x 1 matrix of the corresponding
                             actual values for the measured variables

        Returns: a float that is the MSE between the generated
        data and the actual data
        """
        # find differences
        predictions = self.generate_predictions(inputs)
        diff = predictions - actual_result
        total_error = 0

        height = diff.shape()[0]
        #iterate over nx1 matrix that is composed of the differences
        for row in range(height):
            total_error += diff[row, 0] ** 2
        return total_error / height



def fit_least_squares(input_data, output_data):
    """
    Create a Linear Model which predicts the output vector
```

```
    given the input matrix with minimal Mean-Squared Error.

    inputs:
        - input_data: an n x m matrix
        - output_data: an n x 1 matrix

    returns: a LinearModel object which has been fit to approximately
    match the data
    """
    #intermediate outputs
    transposed = input_data.transpose()
    xt_x = transposed @ input_data
    xt_y = transposed @ output_data
    #formula for dMSE(w) over dw
    weights = xt_x.inverse() @ xt_y
    return LinearModel(weights)

def fit_lasso(param, iterations, input_data, output_data):
    """
    Create a Linear Model which predicts the output vector
    given the input matrix using the LASSO method.

    inputs:
        - param: a float representing the lambda parameter
        - iterations: an integer representing the number of iterations
        - input_data: an n x m matrix
        - output_data: an n x 1 matrix

    returns: a LinearModel object which has been fit to approximately
    match the data
    """
    # helper function defined as instructed
    def soft_threshold(x_val,t_val):
        if(x_val > t_val):
            return x_val-t_val
        elif(x_val < -t_val):
            return x_val+t_val
        return 0

    # establish inital weights
    weights = fit_least_squares(input_data, output_data).get_weights()
    num_weights = weights.shape()[0]

    # save intermediate outputs
    xt_x = input_data.transpose() @ input_data
    xt_y = input_data.transpose() @ output_data

    # iteration to optimize weights
    index = 0
    while index < iterations:
        w_old = []
        for row in range(num_weights):
            w_old.append(weights[row, 0])

```

```
        for jndex in range(num_weights):

            # compute (X^T @ X_j @ w) at 0,0
            dot_product = 0
            for kndex in range(num_weights):
                dot_product += xt_x[jndex, kndex] * weights[kndex, 0]
            #find vals
            a_value = (xt_y[jndex, 0] - dot_product) / xt_x[jndex, jndex]
            b_value = param / (2 * xt_x[jndex, jndex])
            #update weight
            weights[jndex, 0] = soft_threshold(weights[jndex, 0] + a_value, b_value)

        # calculate total change
        delta = 0
        for row in range(num_weights):
            delta += abs(weights[row, 0] - w_old[row])
        # return if change is within tolerance
        if delta < 10 ** (-5):
            return LinearModel(weights)
        # update index
        index += 1

    return LinearModel(weights)


def run_experiment(iterations):
    """
    Using some historical data from 1954-2000, as
    training data, generate weights for a Linear Model
    using both the Least-Squares method and the
    LASSO method (with several different lambda values).

    Test each of these models using the historical
    data from 2001-2012 as test data.

    inputs:
        - iterations: an integer representing the number of iterations to use

    Print out the model's prediction error on the two data sets
    """
    # training data
    train_input  = read_matrix("comp140_analytics_baseball.txt")
    train_output = read_matrix("comp140_analytics_wins.txt")

    # test data
    test_input  = read_matrix("comp140_analytics_baseball_test.txt")
    test_output = read_matrix("comp140_analytics_wins_test.txt")

    # least squares
    least_squares_model = fit_least_squares(train_input, train_output)
    print("Least-Squares:")
    print("Training error: ",
          least_squares_model.prediction_error(train_input, train_output))
    print("Test error: ",
```

```
            least_squares_model.prediction_error(test_input, test_output))

    # lasso with diff lambda vals
    lambdas = [1000, 50000, 100000]
    for param in lambdas:
        lasso_model = fit_lasso(param, iterations, train_input, train_output)
        print("LASSO (lambda={})".format(param))
        print("Training error: ",
              lasso_model.prediction_error(train_input, train_output))
        print("Test error: ",
              lasso_model.prediction_error(test_input, test_output))

```